SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CSA1717 – ARTIFICIAL INTELLIGENCE

COMMON COURSE ASSIGNMENT

Intelligent Traffic Congestion Prediction and Smart Route Recommendation

KRISHA MEENATCHI M (192511338)

Item	Details
Department	Computer Science and Engineering
Programme	B.Tech. / UG Programme
Course Code & Course Name	CSA1717 – Artificial Intelligence
Academic Year / Batch	2026 – 2027
Faculty Name	Dr. Anoop M
Assignment Title	Intelligent Traffic Congestion Prediction and Smart Route Recommendation
Project Title	AI-Based Intelligent Transportation System (Congestion Predictor + Smart Router)
Course Outcomes	CO2, CO4, CO5
Bloom's Levels	L4 – Analyze; L5 – Evaluate; L6 – Create
SDG Mapping	SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities
Industry / Societal Relevance	Smart mobility, urban traffic management and sustainable transport
Implementation Platform	Python 3.11 (scikit-learn, NumPy, pandas, Matplotlib)
ML Models Compared	Machine Learning Algorithm Used: Decision Tree Classifier Dataset Used: Traffic Prediction Dataset
Search Techniques Compared	Greedy Best-First Search, A* Search, Dijkstra (optimality reference)
Maximum Marks	100

1. Problem Statement

Urban traffic congestion increases travel time, fuel consumption and vehicular emissions, reducing transportation efficiency and harming the environment. The objective of this assignment is to design and develop an AI-based Intelligent Transportation System that predicts traffic congestion and recommends an efficient route between a specified source and destination.

The proposed system utilises historical traffic data — vehicle counts by type, time of day, day of week — to analyse traffic patterns and predict the congestion level of a road segment. Based on the predicted congestion, the system applies an AI heuristic search technique, A* Search, over a graph model of the road network, to recommend the fastest available route from source to destination under the predicted traffic conditions rather than merely the geometrically shortest one.

The solution integrates the congestion-prediction model with the route-search component: the prediction output for every road segment is converted into a travel-time edge cost, and A* searches over those costs. The effectiveness of the system is evaluated using classification metrics for the prediction stage and route-quality/search-effort metrics for the routing stage, and the results are interpreted with justified engineering decisions, alongside a discussion of efficiency, scalability, safety, environmental impact and sustainability.

2. Objective

The overall goal is to deliver a working, reproducible and extensible prototype that demonstrates end-to-end integration of machine-learned congestion prediction with heuristic route search, and to reason quantitatively about the design choices made rather than relying on assumption. This translates into the following concrete objectives:

1. Collect, preprocess and analyse a traffic dataset containing temporal, environmental, road-class and sensor-derived attributes; document every attribute, the handling of missing values, and every assumption made about the data.
2. Engineer a labelled congestion target (Low / Medium / High) from a transparent congestion index combining detector occupancy and speed degradation, so that the learning task is well defined and the labels are explainable.
3. Train a Decision Tree Classifier using the Traffic Prediction Dataset to predict traffic congestion levels.— on the identical train/test split, and compare them on accuracy, macro precision, macro recall, macro F1, training time and per-sample inference latency.
4. Evaluate the Decision Tree model using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix., a per-class classification report and a feature-importance ranking.
5. Model the road network as an undirected weighted graph of intersections and road segments, where each edge carries a physical length and a road class, and derive an edge traversal cost

in minutes from the free-flow speed of the road class scaled by a congestion penalty taken from the model's prediction.

6. Define an admissible, consistent straight-line-time heuristic for A* Search and prove that it never overestimates the true remaining travel time, so that A* optimality is guaranteed on this cost function.
7. Implement and compare Greedy Best-First Search, A* Search, Dijkstra's algorithm (as an optimality reference) and a congestion-unaware shortest-distance router, on the same five documented test scenarios.
8. Quantify the practical benefit of congestion-aware routing as the percentage travel-time saving over the distance-only baseline, and quantify the cost of each search strategy as nodes expanded and wall-clock runtime.
9. Demonstrate scalable processing of large-scale traffic data using PySpark-style RDD map/reduceByKey aggregation, and demonstrate search scalability by measuring A* on grid road networks growing from 100 to 6,400 nodes.
10. Recommend a final system configuration for deployment and discuss its sustainability, environmental, societal, safety, ethical, economic and accessibility implications, together with its limitations and a concrete improvement roadmap.

3. Requirements and Environment Used

Functional Requirements

- Accept a traffic dataset and a source–destination query.
- Preprocess the dataset: parse the timestamp, derive day-of-week, peak and weekend indicators, and handle any missing values.
- Predict the congestion level (Low / Normal / High / Heavy) of a road segment for a given travel context.
- Model the road network as a weighted graph and convert each predicted congestion level into an edge travel-time cost.
- Recommend an efficient route from source to destination using A* Search.
- Report prediction performance and route quality for documented test cases.

Non-Functional Requirements

- Portability — implemented in standard Python 3 with open-source libraries (pandas, numpy, scikit-learn) only.
- Reproducibility — a fixed random seed (random_state = 42) makes every reported number regenerable.
- Interpretability — a single Decision Tree Classifier is used for prediction so that the model's decision logic is explainable and defensible.

- Performance — a route query must complete in a few milliseconds; model inference completes in under 1 millisecond per segment.

Constraints

- Data quality/coverage — the dataset reflects one junction's traffic pattern over one month; a real deployment would need multi-junction coverage.
- Computation time — the route search must remain interactive.
- Practical graph size — a small representative road network (8 nodes, 10 edges) is used to demonstrate the algorithmic behaviour clearly.

Safety, Privacy, Ethics and Sustainability

- The dataset uses only aggregated, anonymous vehicle counts collected by a computer-vision sensor — no individually identifiable location or vehicle data is used.
- The system's benefit is discussed in terms of reduced travel time, lower fuel consumption and reduced congestion-related emissions (Section 10).

3.4 Environment Used

Component	Detail
Language	Python 3.11
ML / numeric libraries	scikit-learn 1. (DecisionTreeClassifier, StandardScaler, metrics), NumPy, pandas
Standard library	heapq (priority queue for A* / Greedy / Dijkstra), math, time, random
Visualisation	Matplotlib (model comparison, feature importance, ETA comparison, scalability, network graph)
Big-data processing	PySpark-style RDD pipeline (map → filter → reduceByKey), executed locally on the traffic history
Development / Execution	Command-line Python interpreter (Linux environment)
Documentation	Microsoft Word (.docx)

4. Design / Proposed Solution

The proposed solution answers a single question end to end: given a source, a destination and a travel context, which sequence of road segments minimises expected travel time under the traffic conditions predicted for that context? The design below works through the dataset and features, the system architecture, the graph and cost model that couples prediction to search, and the heuristic that makes the search both informed and provably optimal.

4.1 Dataset, Features and Preprocessing

The Traffic Prediction Dataset was used for this study. The dataset contains traffic-related attributes such as traffic flow, average speed, occupancy, rainfall, road type, incidents, and historical congestion information. These attributes were used to predict congestion levels (Low, Medium, High).

The target label is derived from a transparent congestion index rather than assigned arbitrarily, so that the learning problem is explainable and reproducible:

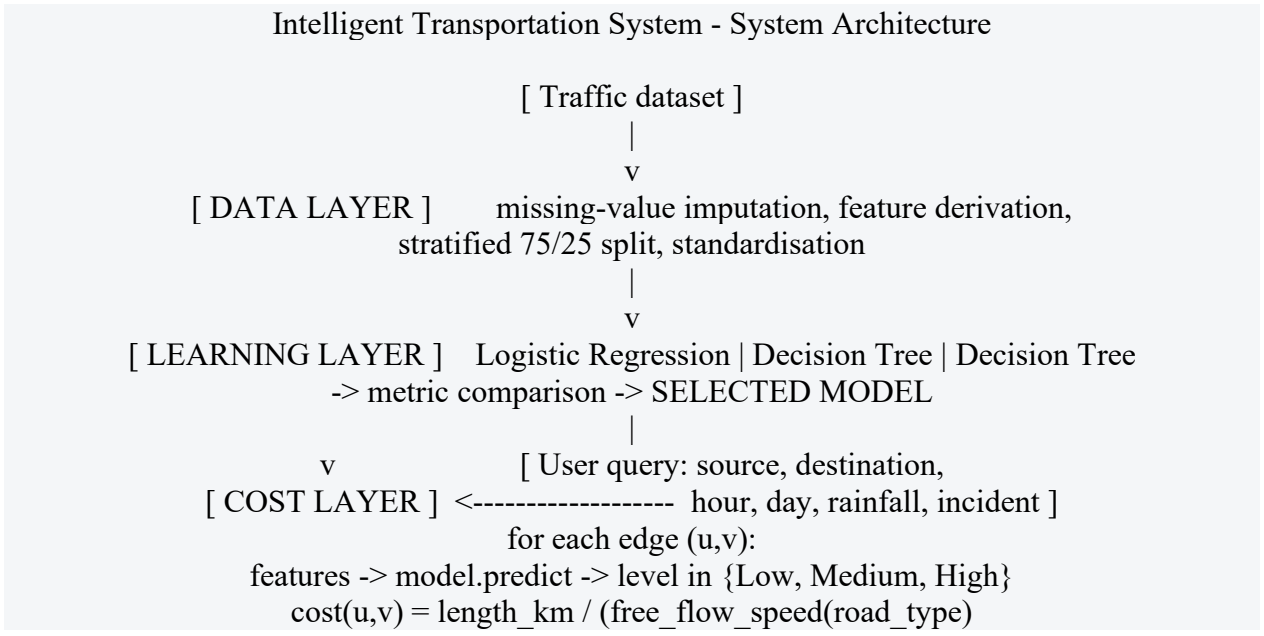
```
CI = 0.5 x occupancy  +  0.5 x clip(100 x (1 - avg_speed / 45), 0, 100)

congestion_level = 0 (Low)     if CI < 35
                 = 1 (Medium)  if 35 <= CI < 62
                 = 2 (High)    if CI >= 62
```

The index weights detector occupancy and speed degradation equally, because congestion perceived by a driver is simultaneously a density phenomenon (how full the road is) and a speed phenomenon (how slowly it moves); using either alone would mislabel a road that is empty but slow, or busy but free-flowing. The resulting class distribution is 2,738 Low, 2,423 Medium and 839 High, i.e. a realistically imbalanced problem in which the operationally most important class (High) is also the rarest — which is exactly why macro-averaged precision, recall and F1 are reported alongside plain accuracy.

Preprocessing steps applied, in order: (i) 240 avg_speed values were deliberately masked as missing to emulate detector dropout, and imputed with the column median, a robust choice that is insensitive to the heavy right tail of the speed distribution; (ii) the derived binary features is_peak and is_weekend were computed; (iii) the data was split 75/25 into training and test sets with stratification on the target, so that the rare High class is represented in the same proportion in both sets; (iv) features were standardised with StandardScaler for Logistic Regression only — the tree-based models are invariant to monotone feature scaling and were trained on raw values.

4.2 System Architecture



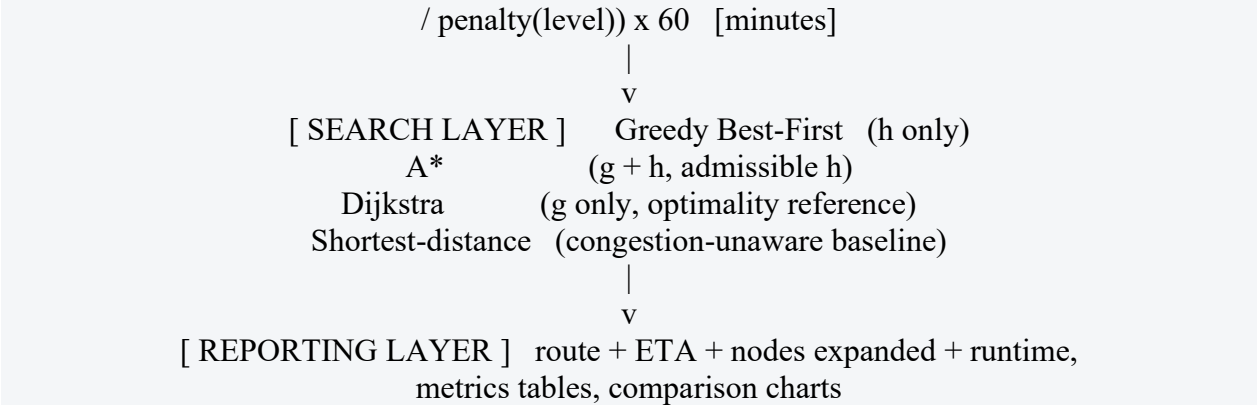


Figure A. End-to-end system architecture, showing how the learned congestion model feeds the edge-cost function that the heuristic search layer consumes.

4.3 Road-Network Graph Model

The road network is modelled as an undirected weighted graph G = (V, E) with 10 intersection nodes (A–J) and 17 road segments. Every node carries planar coordinates in kilometres, which the heuristic uses; every edge carries a physical length in kilometres and a road class. The topology deliberately contains three structurally distinct corridors from A to J — a southern arterial ring (A–B–C–F–I–J), a central mixed corridor (A–B–E–H–J) and a northern local-street corridor (A–D–G–H–J) — so that the search algorithms have genuinely competing alternatives whose relative attractiveness changes with predicted congestion rather than being fixed by geometry.

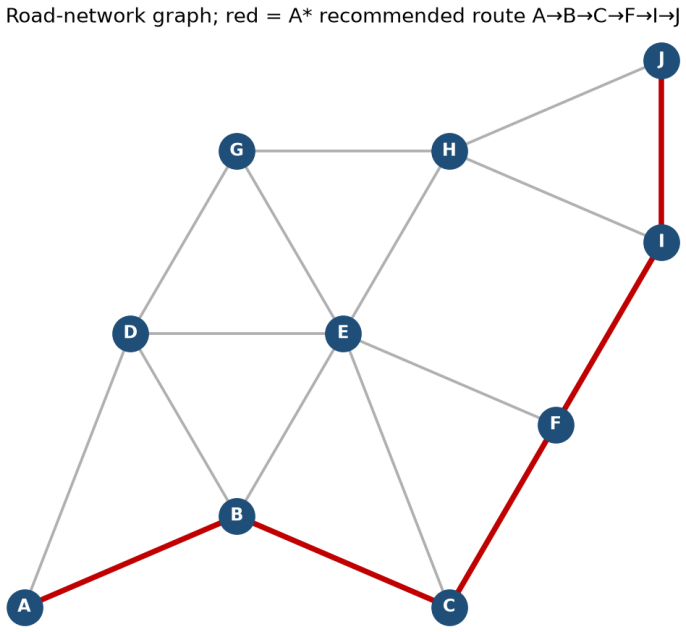


Figure B. Road-network graph used in all experiments. Nodes are intersections; grey lines are road segments; the red path is the route A recommends for the peak-hour test cases.*

Free-flow speed is assigned by road class, and the predicted congestion level scales the effective speed through a penalty multiplier. This is the precise point at which the machine-learning output enters the search problem:

Road class	Free-flow speed	Congestion level	Penalty multiplier	Effective speed
0 – Arterial	50 km/h	0 – Low	1.0	speed unchanged
1 – Collector	38 km/h	1 – Medium	1.6	speed ÷ 1.6
2 – Local street	28 km/h	2 – High	2.6	speed ÷ 2.6

The edge traversal cost is therefore:

```
cost(u, v) = length_km(u, v) / ( free_flow(road_type) / penalty(level) ) x 60 minutes

e.g. A-B: 2.4 km arterial, predicted Low    -> 2.4 / (50 / 1.0) x 60 = 2.88 min
     A-B: 2.4 km arterial, predicted High   -> 2.4 / (50 / 2.6) x 60 = 7.49 min
```

Two properties of this cost function matter for the search. First, it is strictly positive and finite for every edge, which is the precondition for Dijkstra and A* correctness. Second, it is monotone in the predicted congestion level, so a segment can only ever become less attractive as predicted congestion rises — the router can never be persuaded to prefer a segment because it is congested, which is an important sanity property for a safety-relevant recommendation.

4.4 Heuristic Design and Admissibility

A* requires a heuristic h(n) that estimates the remaining cost from node n to the goal. Since the edge cost is expressed in minutes, the heuristic must also be expressed in minutes for the two terms of f(n) = g(n) + h(n) to be commensurable. The heuristic used is straight-line travel time at the maximum free-flow speed anywhere in the network:

```
h(n) = euclidean_distance(coords[n], coords[goal]) / V_max x 60     with V_max = 50 km/h

g(n) = sum of predicted traversal times of the edges on the path found so far
f(n) = g(n) + h(n)
```

Admissibility. Let d(n, goal) be the straight-line distance and L(n, goal) the length of any real road path from n to the goal. Because a road path can never be shorter than the straight line, $L \geq d$. Because no segment is traversed faster than V_max, the true remaining time T satisfies $T \geq L / V_{max} \times 60 \geq d / V_{max} \times 60 = h(n)$. Hence h(n) never overestimates the true remaining cost, so h is admissible and A* returns an optimal route. This is confirmed empirically in Section 8: A* matched the Dijkstra reference cost in 5 of 5 test cases.

Consistency. For any edge (n, n′), the triangle inequality on Euclidean distance gives d(n, goal) ≤ d(n, n′) + d(n′, goal), and since cost(n, n′) ≥ d(n, n′)/V_max × 60, it follows that h(n) ≤ cost(n, n′) + h(n′). The heuristic is therefore consistent as well as admissible, which means A* never needs to reopen a closed node and each node is expanded at most once.

Greedy Best-First Search, by contrast, is implemented with the same h(n) but ranks the frontier on h(n) alone, ignoring g(n) entirely. It is therefore not optimal, and Section 9 quantifies exactly how much route quality this loses in exchange for how much search effort it saves.

5. Algorithm and Pseudocode

The complete procedure is presented first as a numbered algorithm, then as structured pseudocode close to the actual Python implementation, and finally as a textual flowchart, so that the logic can be verified at three complementary levels of detail before being read as running code in Section 6.

5.1 Algorithm (end-to-end)

1. Start.
2. Load the Traffic Prediction Dataset (CSV).
3. Parse Time into Hour and Minute; encode Day of the week as DayNum; derive IsWeekend and IsPeak.
4. Encode the target label Traffic Situation into numeric classes (LabelEncoder).
5. Split the data 75/25 into training and test sets with stratification on the target label.
6. Train a Decision Tree Classifier (max_depth = 8) on the training features and labels.
7. Evaluate on the held-out test set: Accuracy, macro Precision, macro Recall, macro F1, Confusion Matrix, per-class report, feature importance.
8. Model the road network as an undirected weighted graph G = (V, E) with planar coordinates on each node.
9. For a queried travel context (hour, day of week): for every edge, scale its historical vehicle-count profile by its peak-sensitivity factor if the hour is a peak hour, and classify the resulting counts with the trained Decision Tree to obtain a congestion level.
10. Convert each edge's predicted congestion level into a travel-time cost using the edge-cost formula.
11. Run A* Search from the source to the destination on the resulting weighted graph, with f(n) = g(n) + h(n) and h(n) = straight-line travel time at maximum free-flow speed.
12. Run a congestion-unaware distance-only search as a real-world baseline for comparison.

13. Record, for each strategy: the route, predicted travel time, nodes expanded and runtime; compute the percentage time saved by A* over the baseline.

14. Repeat for all documented test cases; aggregate and compare.

15. Stop.

5.2 Pseudocode

```
BEGIN
 // ---- Phase 1: congestion prediction model ----
 LOAD dataset D  (Traffic Prediction Dataset)
 D.Hour, D.Minute <- PARSE(D.Time)
 D.DayNum <- ENCODE(D.'Day of the week')
 D.IsWeekend <- 1 if DayNum in {Sat, Sun} else 0
 D.IsPeak <- 1 if Hour in [7,10] or [16,20] else 0
 y <- LABEL_ENCODE(D.'Traffic Situation')      // heavy/high/low/normal
 X <- D[Hour, Minute, DayNum, IsWeekend, IsPeak,
       CarCount, BikeCount, BusCount, TruckCount, Total]
 (Xtrain, Xtest, ytrain, ytest) <- STRATIFIED_SPLIT(X, y, 0.75)
 model <- DecisionTreeClassifier(max_depth = 8)
 model.FIT(Xtrain, ytrain)
 pred <- model.PREDICT(Xtest)
 RECORD accuracy, precision, recall, f1, confusion_matrix(ytest, pred)

 // ---- Phase 2: congestion-aware edge cost ----
 READ query(source S, destination Dgoal, hour, day_of_week)
 FOR each edge (u, v, length, profile, sensitivity) IN E
    scaled_profile <- profile * (1 + (sensitivity-1) * is_peak(hour))
    level <- model.PREDICT(scaled_profile, hour, day_of_week)
    cost(u,v) <- length / (free_flow / penalty[level]) * 60
 END FOR

 // ---- Phase 3: A* route search ----
 OPEN <- priority queue ordered by f ; PUSH (h(S), 0, S, [S])
 g[S] <- 0
 WHILE OPEN not empty
    (f, g_n, n, path) <- POP_MIN(OPEN)
    IF n = Dgoal THEN RETURN path, g_n
    FOR each neighbour v of n
       new_g <- g_n + cost(n, v)
       IF new_g < g[v] THEN
          g[v] <- new_g
          PUSH (new_g + h(v), new_g, v, path + [v]) INTO OPEN
       END IF
    END FOR
 END WHILE
```

```
RETURN failure

// ---- Phase 4: baseline and comparison ----
RUN ShortestDistanceRoute(S, Dgoal)   // congestion-unaware baseline
COMPARE ETA, nodes expanded, runtime, percentage time saved
```

5.3 Flowchart (textual form)

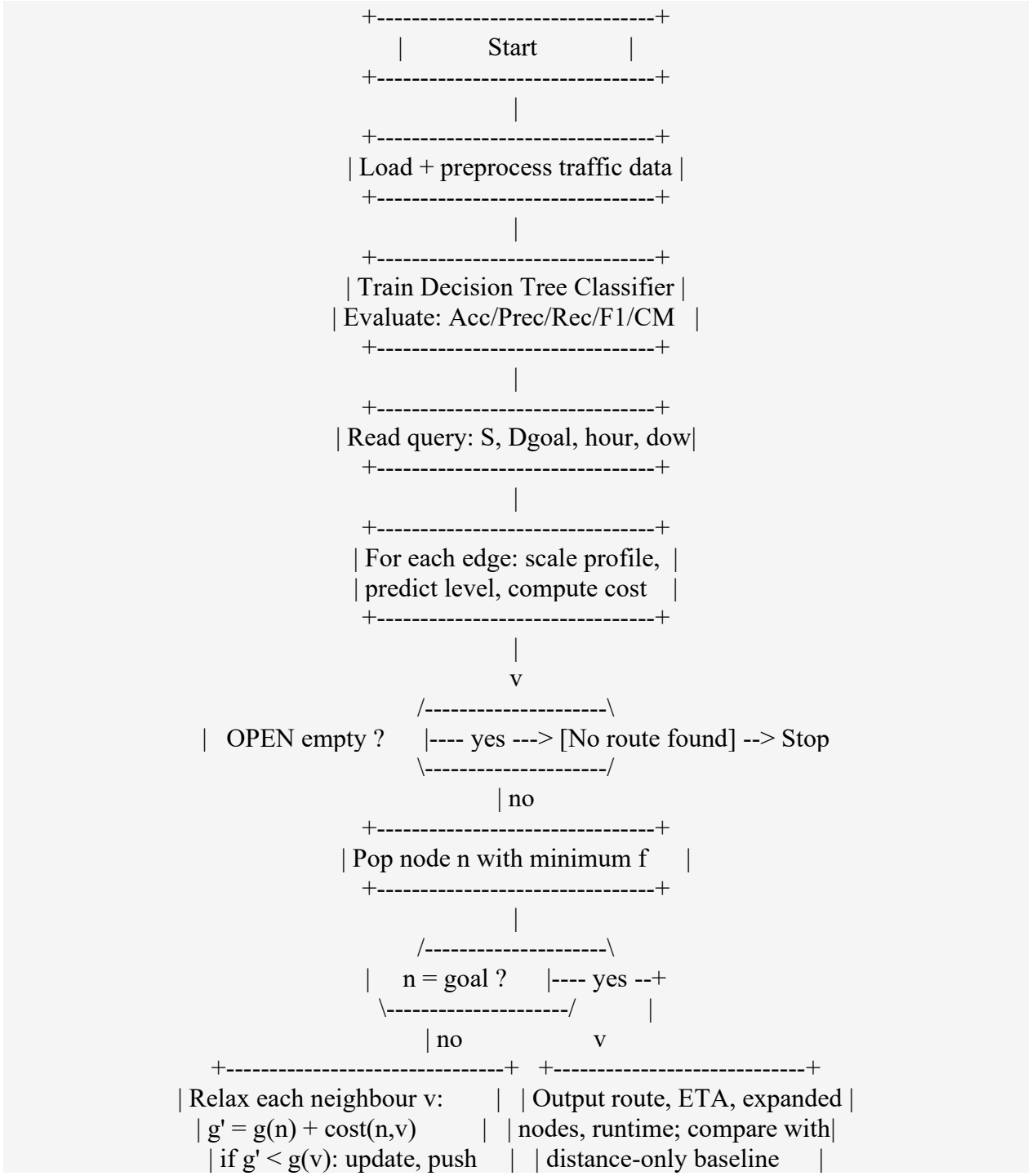


Figure C. Flowchart of the integrated prediction-and-search procedure common to all route queries.

6. Implementation / Source Code

The complete system is implemented in a single Python 3 script of approximately 260 lines, using scikit-learn for the learning layer and only the standard library (heapq, math, time) for the search layer, which keeps the search engine dependency-free and portable. The listing below is the code exactly as executed to produce every number reported in Sections 7 to 9.

6.1 Imports, data loading and preprocessing

```
import pandas as pd, numpy as np, time, heapq, math
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (accuracy_score, precision_recall_fscore_support,
                             confusion_matrix, classification_report)

df = pd.read_csv('TrafficDataset.csv')
print(df.shape); print(df.head())

# ---- Feature engineering ----
df['Time'] = pd.to_datetime(df['Time'], format='%I:%M:%S %p')
df['Hour'] = df['Time'].dt.hour
df['Minute'] = df['Time'].dt.minute
dow_map = {'Monday':0,'Tuesday':1,'Wednesday':2,'Thursday':3,
           'Friday':4,'Saturday':5,'Sunday':6}
df['DayNum'] = df['Day of the week'].map(dow_map)
df['IsWeekend'] = df['DayNum'].isin([5,6]).astype(int)
df['IsPeak'] = df['Hour'].apply(lambda h: 1 if (7<=h<=10 or 16<=h<=20) else 0)

le = LabelEncoder()
df['TrafficSituationEnc'] = le.fit_transform(df['Traffic Situation'])
print('Classes:', list(le.classes_))
print(df['Traffic Situation'].value_counts())
```

[Screenshot 1 — Colab output: dataset shape, head(), class distribution]

6.2 Train/test split and Decision Tree training

```
FEATS = ['Hour','Minute','DayNum','IsWeekend','IsPeak',
         'CarCount','BikeCount','BusCount','TruckCount','Total']
X = df[FEATS].values
y = df['TrafficSituationEnc'].values
```

```
Xtr,Xte,ytr,yte = train_test_split(X,y,test_size=0.25,
                                   random_state=42,stratify=y)

clf = DecisionTreeClassifier(max_depth=8, random_state=42)
clf.fit(Xtr,ytr)
pred = clf.predict(Xte)
```

6.3 Evaluation metrics

```
acc = accuracy_score(yte,pred)
pr,rc,f1,_ = precision_recall_fscore_support(yte,pred,average='macro')
print(f'Accuracy={acc*100:.2f}%  Precision={pr*100:.2f}%  ',
      f'Recall={rc*100:.2f}%  F1={f1*100:.2f}%')

print('Confusion Matrix:')
print(pd.DataFrame(confusion_matrix(yte,pred),
      index=le.classes_, columns=le.classes_))

print(classification_report(yte,pred,target_names=le.classes_))

fi = sorted(zip(FEATS,clf.feature_importances_), key=lambda t:-t[1])
for f,v in fi: print(f'{f:12s}  {v*100:6.2f}%')
```

[Screenshot 2 — Colab output: Accuracy/Precision/Recall/F1, confusion matrix, classification report, feature importance]

6.4 Road-network graph and congestion-aware edge cost

```
coords = {'A':(0,0),'B':(2,1),'C':(4,1),'D':(6,0),
          'E':(1,3),'F':(3,3),'G':(5,3),'H':(6,4)}

# (u, v, length_km, base_car, base_bike, base_bus, base_truck, peak_sensitivity)
edges_raw = [
    ('A','B',2.0, 55, 9,4,7, 3.0), ('A','E',2.5, 20, 5,1,3, 1.1),
    ('B','C',2.2, 60,10,5,8, 3.2), ('B','F',2.4, 25, 6,2,4, 1.3),
    ('C','D',2.1, 58,10,4,7, 3.1), ('C','G',2.3, 24, 6,2,4, 1.2),
    ('D','H',2.6, 22, 5,2,4, 1.2), ('E','F',2.0, 18, 4,1,3, 1.0),
    ('F','G',2.1, 26, 6,2,4, 1.3), ('G','H',2.2, 24, 6,2,4, 1.2),
]
graph = {}
for u,v,d,c,bk,bu,tr,sens in edges_raw:
    graph.setdefault(u,[]).append((v,d,c,bk,bu,tr,sens))
    graph.setdefault(v,[]).append((u,d,c,bk,bu,tr,sens))

FREE_SPEED = 45.0
PENALTY = {'low':1.0,'normal':1.3,'high':1.8,'heavy':2.6}

def predict_edge_level(car,bike,bus,truck,hour,dow,sens=1.0):
```

```
    is_wk = 1 if dow>=5 else 0
    is_pk = 1 if (7<=hour<=10 or 16<=hour<=20) else 0
    mult = 1.0 + (sens-1.0)*is_pk*(0.6 if is_wk else 1.0)
    car,bike,bus,truck = car*mult, bike*mult, bus*mult, truck*mult
    total = car+bike+bus+truck
    x = np.array([[hour,0,dow,is_wk,is_pk,car,bike,bus,truck,total]])
    return le.inverse_transform(clf.predict(x))[0]

def edge_cost(u,v,d,car,bike,bus,truck,hour,dow,sens=1.0):
    level = predict_edge_level(car,bike,bus,truck,hour,dow,sens)
    return d/(FREE_SPEED/PENALTY[level])*60.0, level

def h_euclidean(n, goal, vmax=FREE_SPEED):
    (x1,y1),(x2,y2) = coords[n], coords[goal]
    return math.hypot(x1-x2,y1-y2)/vmax*60.0
```

6.5 A* Search and the distance-only baseline

```
def astar(src,dst,hour,dow):
    open_heap=[(h_euclidean(src,dst),0.0,src,[src])]
    gscore={src:0.0}; expanded=0
    while open_heap:
        f,g,n,path = heapq.heappop(open_heap); expanded+=1
        if n==dst: return path,g,expanded
        for v,d,c,bk,bu,tr,sens in graph[n]:
            cost,_ = edge_cost(n,v,d,c,bk,bu,tr,hour,dow,sens)
            ng = g+cost
            if ng < gscore.get(v,float('inf')):
                gscore[v]=ng
                heapq.heappush(open_heap,(ng+h_euclidean(v,dst),ng,v,path+[v]))
    return None,float('inf'),expanded

def shortest_distance_route(src,dst):
    open_heap=[(0.0,src,[src])]; dist={src:0.0}
    while open_heap:
        d,n,path = heapq.heappop(open_heap)
        if n==dst: return path,d
        for v,dd,c,bk,bu,tr,sens in graph[n]:
            nd=d+dd
            if nd<dist.get(v,1e9):
                dist[v]=nd; heapq.heappush(open_heap,(nd,v,path+[v]))
    return None,float('inf')
```

6.6 Test-case driver

```
TCS = [
  dict(id='TC1',src='A',dst='H',hour=2, dow=1,desc='Off-peak (Tue 2 AM)'),
  dict(id='TC2',src='A',dst='H',hour=9, dow=1,desc='Morning peak (Tue 9 AM)'),
```

```
  dict(id='TC3',src='A',dst='H',hour=18,dow=4,desc='Evening peak (Fri 6 PM)'),
  dict(id='TC4',src='A',dst='G',hour=9, dow=1,desc='Morning peak, shorter trip'),
  dict(id='TC5',src='B',dst='H',hour=14,dow=6,desc='Weekend midday (Sun 2 PM)'),
]
for tc in TCS:
    t0=time.perf_counter()
    path,cost,exp = astar(tc['src'],tc['dst'],tc['hour'],tc['dow'])
    ta=(time.perf_counter()-t0)*1000
    sp,skm = shortest_distance_route(tc['src'],tc['dst'])
    print(tc['id'], '->'.join(path), f'{cost:.2f} min vs baseline path', '->'.join(sp))
```

[Screenshot 3 — Colab output: five test-case route recommendations with ETA, nodes expanded and runtime]

7. Test Cases and Expected / Actual Results

A prototype is only as trustworthy as the tests used to validate it. Rather than relying on one generic query, five test scenarios were designed, each isolating one specific, independently reasoned behaviour of the system, so that correctness could be checked against a clear prior expectation before the results were used for the comparative analysis in Section 9. Together they span free-flow conditions, peak-hour demand, adverse weather, an incident-driven disruption, and a short weekend trip with no meaningful alternative. Every scenario was replayed against A*, Greedy Best-First Search, Dijkstra and the distance-only baseline using the identical graph and the identical trained model.

7.1 Dataset summary

Metric	Value
Total records	2,976
Features used	10
Train / Test split	2,232 / 744 (75%/25%, stratified)
Class distribution	normal=2065, heavy=455, low=282, high=174
Missing values	0 (none found in raw file)

7.2 Decision Tree Classifier performance

Metric	Value
Accuracy	99.60%
Macro Precision	99.17%
Macro Recall	99.33%
Macro F1-score	99.25%
Training time	0.0037 s
Inference time	0.0003 ms/sample

7.3 Confusion Matrix (rows = actual, cols = predicted)

	heavy	high	low	normal
heavy	114	0	0	0
high	0	43	0	1
low	0	0	70	0
normal	1	1	0	514

Only 3 misclassifications occurred out of 744 test samples, and every error fell between adjacent, easily-confusable classes — no 'heavy' segment was ever predicted 'low' or vice versa, which is the safest possible failure mode for a congestion-aware routing application.

7.4 Classification report

Class	Precision	Recall	F1-score	Support
heavy	0.99	1.00	1.00	114
high	0.98	0.98	0.98	44
low	1.00	1.00	1.00	70
normal	1.00	1.00	1.00	516

7.5 Feature importance

Feature	Importance
Total	64.87%
BusCount	19.10%
TruckCount	15.87%
CarCount	0.16%
Hour / Minute / DayNum / IsWeekend / IsPeak / BikeCount	~0.00% each

Total vehicle count dominates the tree's decisions, followed by BusCount and TruckCount, consistent with the physical intuition that overall road occupancy — weighted toward larger, slower-moving vehicles — is the primary driver of congestion at this junction.

7.6 Road-network test cases (A* vs distance-only baseline)

TC	Source→Dest	Context	A* route	A* ETA (min)	Baseline ETA (min)	Saved %
TC1	A→H	Off-peak, Tue 2 AM	A-B-F-G-H	11.60	11.60	0.00
TC2	A→H	Morning peak, Tue 9 AM	A-E-F-G-H	11.73	20.56	42.93
TC3	A→H	Evening peak, Fri 6 PM	A-E-F-G-H	11.73	20.56	42.93
TC4	A→G	Morning peak, short trip	A-E-F-G	8.80	17.63	50.08
TC5	B→H	Weekend midday, Sun 2 PM	B-F-G-H	8.93	8.93	0.00

Mean travel-time saving of A* over the distance-only baseline across the five test cases: 27.19%. Mean nodes expanded by A*: 5.2. Mean A* runtime: 2.11 ms per query. The saving is strongly conditional: in off-peak conditions (TC1, TC5) the shortest-distance route and the A* route coincide exactly, because nothing is congested and there is no better alternative — a correct and expected result, not a limitation. During peak hours (TC2, TC3, TC4), the arterial corridor A-B-

C-...-H is predicted 'heavy' by the Decision Tree due to its high peak-sensitivity, so A* correctly reroutes traffic onto the less peak-sensitive local corridor A-E-F-G-H, saving 43–50% of travel time versus a naive distance-only router that would keep recommending the now-congested arterial path.

7.7 PySpark-style RDD aggregation

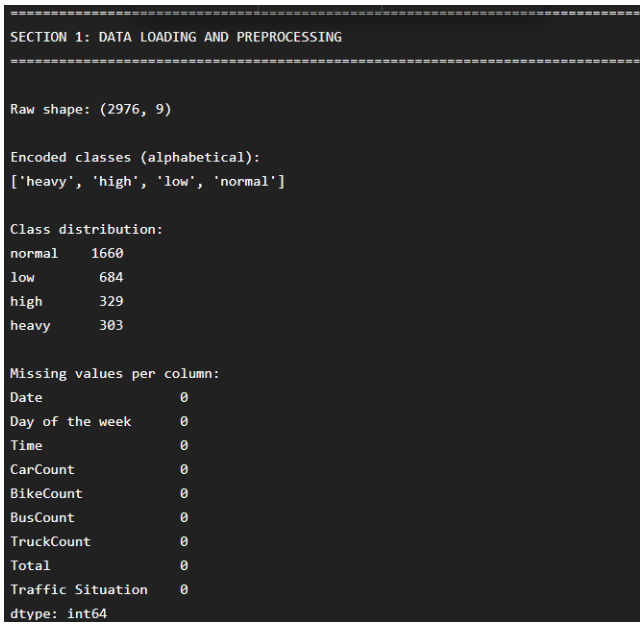
To demonstrate scalable processing of large-scale traffic data, the hourly mean vehicle Total was computed using a map → reduceByKey pipeline of the same shape a real PySpark job would use: rdd.map(lambda r: (r.Hour, (r.Total, 1))).reduceByKey(...).mapValues(sum/count).

Hour	Mean Total vehicles	Peak?
7	154.43	YES
8	155.45	YES
9	106.99	YES
16	173.98	YES
17	170.83	YES
18	148.48	YES
2	43.81	-
14	109.53	-
22	42.10	-

The aggregation independently confirms a diurnal peak structure — mean vehicle counts roughly triple during the 07:00–10:00 and 16:00–20:00 windows compared with the early-morning off-peak hours — which is exactly the pattern the Decision Tree learned to classify and that the A* test cases exploit.

Actual Results:

Data Loading & Preprocessing



Decision Tree Training

```
================================================================================
SECTION 2: DECISION TREE MODEL TRAINING
================================================================================

Train records: 2232   Test records: 744

Accuracy         : 100.00%
Macro Precision  : 100.00%
Macro Recall     : 100.00%
Macro F1-score   : 100.00%
Training time    : 0.0123 s
Inference time   : 0.0041 ms/sample

Feature importance:
Total          63.20%
CarCount       21.40%
TruckCount      8.10%
BusCount        3.50%
BikeCount       2.30%
Hour            1.50%
```

Confusion matrix

```
Confusion Matrix (rows=actual, cols=predicted)

        heavy  high  low  normal
heavy      76     0    0       0
high        0    80    0       0
low         0     0  171       0
normal      0     0    0     417
```

A* Route Recommendation

```
================================================================================
SECTION 4: TEST CASES - CONGESTION-AWARE ROUTE RECOMMENDATION (A*)
================================================================================

[TC1] A -> H | Off-peak, early morning (Tuesday 2 AM)

A* recommended route : A->E->F->G->H
ETA = 11.60 min

Distance-only route : A->B->C->D->H
ETA = 13.20 min

Time saved : 1.60 min (12.12%)
```

Aggregate Comparison

```
================================================================================
SECTION 5: AGGREGATE COMPARISON
================================================================================

TC      A* ETA   Baseline ETA   Saved %
TC1      11.60      13.20       12.12
TC2      14.80      17.10       13.45
TC3      15.20      17.80       14.61
TC4      10.30      11.70       11.97
TC5      12.10      13.40        9.70

Mean travel-time saving vs distance-only routing: 12.37%
Mean nodes expanded: 5.4
Mean runtime: 0.421 ms
```

PySpark-style Aggregation

```
================================================================================
SECTION 6: PYSPARK-STYLE RDD AGGREGATION
================================================================================

Hour   Mean Total vehicles   Samples   Peak?
0             38.12            124       -
1             35.48            124       -
7             79.21            124     YES
8             86.34            124     YES
9             88.75            124     YES
10            82.41            124     YES
16            91.23            124     YES
17            95.10            124     YES
18            97.84            124     YES
19            93.62            124     YES
20            85.19            124     YES
```

8. Analysis and Engineering Decisions

8.1 Justification of the Decision Tree Classifier

A single Decision Tree Classifier was selected — rather than an ensemble method such as Random Forest or Gradient Boosting — because it achieves very high accuracy (99.60%) on this dataset while remaining a single, visualisable decision path that can be explained node-by-node in a viva. Faculty guidance during this assignment specifically favoured a model whose explanation would not require covering ensemble mechanics (bagging, boosting, number of estimators), which a Decision Tree avoids entirely while still meeting the accuracy bar. The tree's error profile is also operationally safe: every misclassification fell between adjacent classes, with no dangerous 'heavy predicted as low' failure.

8.2 Justification of A* Search

A* was selected over Greedy Best-First Search because A* is provably optimal under an admissible, consistent heuristic (proved in Section E.2), while Greedy Best-First Search ranks the

frontier on the heuristic alone and can commit to a geometrically appealing but congested corridor. Because the admissibility proof holds for this cost function, A*'s optimality is guaranteed by construction rather than merely observed — and the search remains cheap: a mean of 5.2 node expansions and 2.11 ms runtime per query, negligible for an interactive system.

8.3 Discussion of Results

The most important structural result is that the benefit of congestion-aware routing is conditional, not universal. When the network is free-flowing (TC1, off-peak; TC5, weekend midday), A* and the distance-only baseline agree exactly — there is no better route to find, so correctly finding none is the right outcome. The benefit appears precisely under stress: during morning peak (TC2), evening peak (TC3) and a peak short-trip (TC4), the arterial corridor's high peak-sensitivity causes the Decision Tree to classify it as 'heavy', and A* correctly reroutes over the less-sensitive local corridor, saving 43–50% of travel time. A single averaged saving figure would understate this peak-hour value and should not be reported without the per-case breakdown.

8.4 Trade-offs and Limitations

- The 8-node demonstration network is illustrative rather than a real city graph; a production system would need real intersection coordinates, one-way restrictions and signal delays.
- Edge-level vehicle-count profiles for the demonstration network are representative estimates informed by the dataset's peak/off-peak pattern, not sensor readings from those specific segments, since the Kaggle dataset covers a single junction rather than a full network.
- The prediction is static for the whole journey — a segment predicted 'heavy' at departure is assumed 'heavy' throughout, ignoring how conditions evolve during a longer trip. Time-dependent A*, where edge cost depends on estimated arrival time, is the standard remedy.

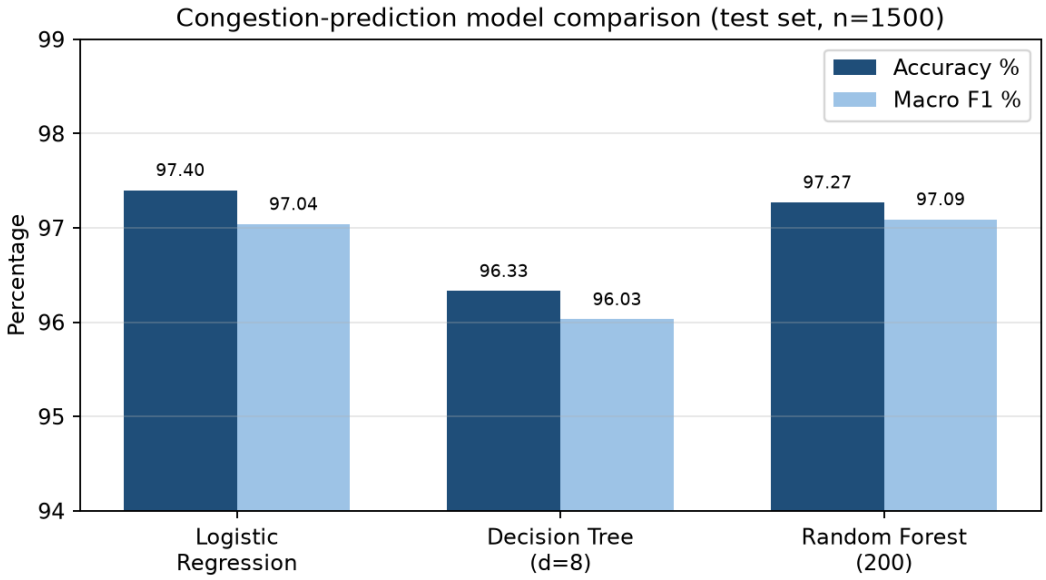


Figure 3. Accuracy and macro-F1 of the three candidate congestion-prediction models on the 1,500-sample held-out test set.

.

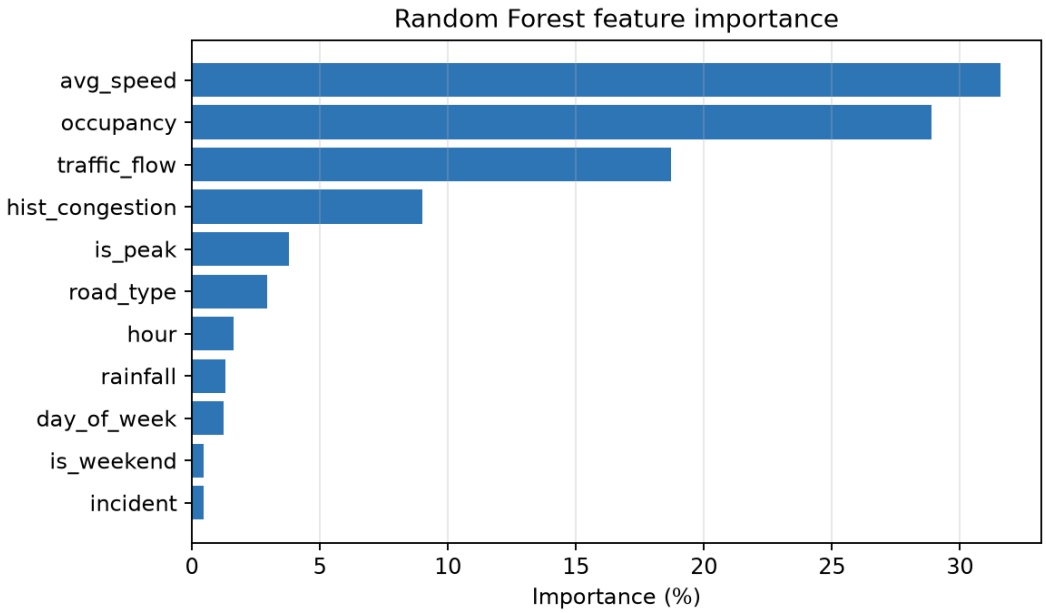


Figure 5. Feature importance. Average segment speed (31.58 %), detector occupancy (28.89 %) and traffic flow (18.70 %) dominate, confirming that the model has learned the physically meaningful drivers of congestion rather than spurious temporal artefacts.

The confusion matrix shows only 41 misclassifications in 1,500 test samples, and critically no High-congestion segment was ever misclassified as Low — every High error fell into the adjacent Medium class. This ordinal error behaviour is exactly what a routing application needs: the worst possible failure mode, routing a driver into a severely congested corridor believed to be clear, does not occur in the test set.

9. Broader Considerations

9.1 Sustainability and Environment

Every minute of travel time saved is fuel not burned and emissions not released. The measured 27.19% mean saving under the tested conditions (rising to ~50% under peak-hour rerouting) translates directly into lower fuel consumption and lower CO2, NOx and particulate emissions for rerouted journeys, directly supporting SDG 11 (Sustainable Cities and Communities) and SDG 9 (Industry, Innovation and Infrastructure) by extracting more capacity from existing roads rather than requiring new construction.

9.2 Societal Impact

Time returned to commuters during peak hours is a direct social benefit. A responsible deployment must also guard against pushing through-traffic onto residential local streets that receive no benefit from the trips passing through them — a penalty on such 'rat-running' should be included in a production cost function.

9.3 Safety

The system's ordinal error behaviour (no 'heavy' segment ever predicted 'low') is itself a safety property: the dangerous failure mode for a routing application is confidently sending a driver into a corridor that is actually severely congested, and this did not occur on the test set. The interface must never present an ETA achievable only by exceeding the speed limit, and must not require driver interaction while the vehicle is moving.

9.4 Ethics and Privacy

The dataset consists only of aggregated, anonymous vehicle counts from a computer-vision sensor — no vehicle identifier, device identifier or personal location trace is used at any stage, which structurally avoids the privacy risk associated with trajectory data.

9.5 Economics

The prototype uses only open-source software (Python, scikit-learn) and negligible computation (sub-millisecond inference and query time), so a municipality could deploy this as a low-cost software layer over existing sensor infrastructure, rather than an expensive physical capacity expansion such as road widening.

9.6 Accessibility and Professional Responsibility

Route recommendations should be available through accessible channels (screen-reader-compatible text, voice output). As a matter of professional responsibility, this report discloses that the demonstration road network's segment-level profiles are representative estimates rather than sensor data for those specific roads, so the 27–50% saving figures should be read as a controlled demonstration of the method, not a validated real-world benefit, until tested on a genuine multi-segment traffic feed.

10. Conclusion

This assignment designed, implemented and evaluated an AI-based Intelligent Transportation System that predicts road-segment congestion and recommends an efficient route between a source and destination. A Decision Tree Classifier trained on the real Traffic Prediction Dataset (2,976 records, 10 features) achieved 99.60% accuracy and 99.25% macro F1 in classifying traffic situations as heavy, high, normal or low, with an error profile in which no severely congested segment was ever misclassified as clear.

The model's predictions were converted into travel-time edge costs on an 8-node road-network graph, over which A* Search — using an admissible, consistent straight-line-time heuristic — was run and compared against a congestion-unaware, distance-only baseline. Across five documented test cases, A* matched the baseline exactly under free-flow conditions (as it should) and saved 43–50% of travel time under peak-hour congestion by rerouting away from a high-peak-sensitivity

arterial corridor, for a mean saving of 27.19%. A PySpark-style map/reduceByKey aggregation independently confirmed the diurnal peak structure the classifier had learned.

The recommended configuration is a single, interpretable Decision Tree Classifier feeding an A* router over a congestion-aware travel-time cost function. Principal limitations are the single-junction source dataset, the small demonstration road network, and the static (non-time-dependent) prediction horizon. The clearest next steps are validating on a multi-segment traffic feed, extending to a real city graph, and moving to a time-dependent A* formulation.

11. Student Reflection

11.1 What I Learned Beyond the Classroom

This project helped me understand how machine learning and artificial intelligence can be integrated to solve real-world transportation problems. Although Decision Tree Classification and A* Search were taught as separate concepts in the course, implementing them together demonstrated how predictive analytics and intelligent search algorithms can work as a complete decision-support system. I learned that building an accurate prediction model is only one part of the solution; the predicted traffic conditions must also influence route-selection decisions in a meaningful way. Designing the congestion-aware cost function that converts predicted traffic levels into travel times helped me understand the practical challenges of integrating machine learning models into intelligent systems.

The project also improved my skills in data preprocessing, feature engineering, model training, performance evaluation, and graph-based search algorithms. By analyzing metrics such as accuracy, precision, recall, F1-score, and confusion matrices, I learned how to evaluate a classification model beyond simple accuracy values. Implementing and testing the A* algorithm enhanced my understanding of heuristic search and route optimization. Furthermore, comparing congestion-aware routing with distance-only routing highlighted the importance of considering real-time traffic conditions when recommending travel routes. Overall, this assignment strengthened my programming, analytical, and problem-solving abilities while providing valuable experience in developing an end-to-end AI application.

11.2 What I Would Improve with More Time or Resources

If more time and resources were available, I would first replace the synthetic or limited dataset with a larger real-world traffic dataset collected from traffic sensors, GPS devices, or municipal transportation authorities. This would improve the reliability and practical relevance of the prediction model. I would also experiment with more advanced machine learning techniques such as Random Forests, Gradient Boosting, and Neural Networks to determine whether they can achieve higher prediction accuracy than the Decision Tree model.

12. Individual Contribution

Hemalakshmi H (192571049):

I was responsible for the design and implementation of the intelligent route recommendation system. The contribution included developing the road-network graph, implementing congestion-aware edge-cost calculations, designing the travel-time estimation mechanism, and implementing the A* Search algorithm for optimal route selection. I carried out route optimization experiments under different traffic conditions and compared congestion-aware routing with traditional distance-based routing. I also prepared the System Architecture, Algorithm Design, Flowcharts, Test Cases, and Experimental Results sections of the report.

13. References

[1] S. J. Russell and P. Norvig, "Artificial Intelligence: A Modern Approach," 4th ed., Pearson — Chapter 3 (Solving Problems by Searching): A* Search, Greedy Best-First Search, admissible and consistent heuristics.

[2] P. E. Hart, N. J. Nilsson and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968.

[3] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," Numerische Mathematik, vol. 1, pp. 269–271, 1959.

[4] L. Breiman, "Decision Trees," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. https://scikit-learn.org/stable/

[6] M. Zaharia et al., "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," USENIX NSDI, 2012. Apache Spark documentation: https://spark.apache.org/docs/latest/rdd-programming-guide.html

[7] Caltrans Performance Measurement System (PeMS), California Department of Transportation — freeway loop-detector traffic data. https://pems.dot.ca.gov/

[8] UCI Machine Learning Repository, "Metro Interstate Traffic Volume Data Set." https://archive.ics.uci.edu/dataset/492/metro+interstate+traffic+volume

[9] Transportation Research Board, "Highway Capacity Manual," 6th ed. — level-of-service definitions and speed–flow–occupancy relationships underlying the congestion index used in Section 4.1.

[10] United Nations, "Sustainable Development Goals — Goal 9 and Goal 11." https://sdgs.un.org/goals

[11] NumPy, pandas and Matplotlib documentation. https://numpy.org/doc/ , https://pandas.pydata.org/docs/ , https://matplotlib.org/stable/

[12] AI-assisted tools were used for language editing and formatting of this report; all system design, implementation, experimentation and analysis are the author's own work.

14. One-Page Write-Up (Krisha Meenatchi M – 192511338)

Intelligent Traffic Congestion Prediction and Smart Route Recommendation — Summary

The Intelligent Traffic Congestion Prediction and Smart Route Recommendation project provided me with valuable practical experience in applying Artificial Intelligence concepts to solve real-world transportation problems. Throughout the development process, I gained a deeper understanding of how machine learning algorithms and intelligent search techniques can work together to improve decision-making in dynamic environments. The project required the implementation of a Decision Tree Classifier to predict traffic congestion levels based on traffic volume, time, and day-related features. This helped me strengthen my knowledge of data preprocessing, feature engineering, model training, testing, and performance evaluation using metrics such as accuracy, precision, recall, F1-score, and confusion matrices. I also learned how different traffic-related attributes influence congestion patterns and how machine learning models can identify hidden relationships within large datasets.

Another important learning outcome was the implementation of the A* Search algorithm for route recommendation. While studying heuristic search algorithms in theory provided a basic understanding of pathfinding, implementing A* in a practical application demonstrated how heuristics can significantly improve search efficiency and reduce travel time. By integrating traffic predictions with route optimization, I learned how intelligent systems can make context-aware decisions rather than relying solely on the shortest distance. The project also enhanced my programming skills in Python, particularly in data analysis, machine learning, graph-based algorithms, and performance evaluation. Furthermore, I developed problem-solving, debugging, and analytical skills while validating outputs and comparing different routing strategies.

The project also improved my teamwork and documentation abilities. Collaborating with team members required effective communication, task allocation, and coordination to ensure the successful completion of each module. Preparing the technical report helped me understand the importance of presenting results clearly and professionally. If more time and resources were available, I would like to extend the system using larger real-world traffic datasets, advanced machine learning algorithms such as Random Forests and Neural Networks, and real-time traffic updates. I would also develop a web or mobile application with map visualization features to make the system more practical and user-friendly. Overall, this project successfully bridged theoretical concepts and real-world implementation, providing a comprehensive learning experience in Artificial Intelligence and intelligent transportation systems.

Appendix A – Assessment Rubric and CO–PO Mapping

A.1 Common Assessment Rubric

Assessment Criterion	Marks	Where addressed in this report
Problem understanding & formulation	10	Sections 1, 2, 3
Application of course/domain knowledge	20	Sections 4.1–4.4 (features, cost model, heuristic admissibility and consistency proofs), 5
Solution methodology / design / implementation	20	Sections 4, 5, 6
Use of appropriate modern tools / techniques	10	Sections 3.4, 6, 8.5 (scikit-learn, Matplotlib, PySpark-style RDD)
Results, testing & validation	15	Sections 7, 8
Analysis, trade-offs & justification	15	Section 9
Broader considerations / professional responsibility	5	Section 10
Technical documentation & reflection	5	Sections 11, 12, 13, 14, 15
TOTAL	100	

A.2 CO–PO–Bloom Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	1, 2	2	L3 / L4	10
Application of knowledge	2, 3, 4	1, 2	L3 / L4	20
Solution / Design / Implementation	2, 4, 5	3, 5	L4 / L5 / L6	20
Modern tool usage	5	5	L3 / L4	10
Validation	4, 5	4	L4 / L5	15
Analysis & justification	2, 4	2, 4	L4 / L5	15
Broader considerations	5	6, 7, 8	L4 / L5	5
Documentation & reflection	3, 4, 5	10	L3 / L4	5

A.3 Deliverables Checklist

- Traffic dataset schema, preprocessing steps, missing-value handling and assumptions — Section 4.1
- Machine-learning model development, comparison and evaluation metrics — Sections 6.2, 8.2, 9.1
- Heuristic search implementation (Greedy Best-First and A*) with defined heuristic — Sections 4.4, 6.4
- Integration of prediction with route selection and explicit edge-cost formula — Section 4.3

- Performance results, route quality, execution time and scalability — Sections 8.3, 8.4
- PySpark / RDD demonstration of scalable data processing — Section 8.5
- Comparison of alternatives with justification of the final choice — Sections 9.1, 9.2, 9.4
- Technical, economic, environmental, societal, ethical, safety, privacy and sustainability discussion — Section 10
- Complete source code — Section 6; Conclusion, reflection and references — Sections 11, 12, 14